

ZAP and HTTP/1.1 on Loopback: An Allocation and Wire-Size Baseline, and a Correction

ZAP Protocol Authors
zap-proto.io

2026-09-10

Abstract

We measure the ZAP wire against HTTP/1.1 on a loopback echo workload at five payload sizes. Five arms answer the identical request: Go’s `net/http`, `fasthttp` over HTTP/1.1, ZAP-HTTP (`fasthttp` objects on the ZAP wire), a native ZAP typed message built with the published `zap-proto/go` runtime, and a length-prefixed byte echo that bounds the others from below. The `fasthttp` arm is what makes the result readable: its object model is far cheaper than `net/http`’s on *any* wire, so a ZAP-against-`net/http` ratio moves for two reasons at once. Held against that matched baseline, ZAP-HTTP is allocation-free in steady state and allocates fewer bytes than `fasthttp` at every size (2.8 vs. 6.7 B/req at 16-byte bodies, 68.5 vs. 72.5 at 64 KiB), and its frame is +0.1–+13.5% larger than the equivalent HTTP/1.1 request rather than the 86% larger the previous revision of this paper measured. The native arm is the correction. This paper previously reported that native ZAP allocates exactly its payload in exactly one allocation per request, up to $622\times$ less memory than `net/http`; those figures came from an arm with no stored output whose implementation was a bespoke TCP byte echo importing nothing from any ZAP package. Rebuilt on `zap-proto/go` v1.8.3 and re-measured, a native round trip allocates *six* objects and two heap copies of the payload, because `ObjectBuilder.SetBytes` heap-copies the payload before copying it into the message buffer. At 64 KiB that is 131 268 B/req against `fasthttp`’s 72.5. The stored run reports *no* throughput: the harness refuses to time anything on a machine whose load average exceeds its core count, and this one began at 102.2 on ten cores. The sequential rates it does print vary by a factor of 5.5 between repetitions of the identical loop, which is what that refusal is for. We print no rate rather than one taken under that. Every number here is a line in `bench-results.txt` at [zap-proto/bench](https://zap-proto.io/bench); this paper’s tables and quoted figures are generated from that file rather than typed, so a column cannot outlive the output behind it.

1 What this paper previously claimed, and why it changed

The previous revision reported that native ZAP allocates “exactly what it sends” — a 16-byte payload producing 16 bytes of allocation — in exactly one allocation per request, for $622\times$ less memory and $115\times$ fewer allocations than `net/http`, at 1.85–3.68 $\times$ its throughput. Three things were wrong with that, and it is worth being precise about which.

The arm had no stored output. The harness commits its raw output to `bench-results.txt`. That file was written by the initial commit (b468d54) at 20:33:54; the native arm arrived eight minutes later (a9766c4, “native-ZAP RPC bench + memory pressure test”) at 20:41:04, and the file was never regenerated. Every number in the stored output came from a two-arm run — HTTP/1.1+JSON and ZAP-binary — in which the string *Native* does not appear. The third column

of every table in the previous revision had no line behind it. (*Hashes are as of this writing; that history has been rewritten once, so the commit messages and times identify them more durably than the hashes do.*)

The arm was not ZAP. More seriously, the code that would have produced those numbers was a hand-written TCP server that read a 4-byte length prefix and wrote the same bytes back without looking at them. It imported no ZAP package and carried no message format, no schema, and no headers, while the HTTP arm it was compared against constructed a request, parsed a header set, and returned a JSON-shaped body. The paper described this arm as “length-prefixed frames, schema-typed bodies”. It had no typed field of any kind. Re-running it would have produced numbers, but they would have measured the cost of copying bytes, not the cost of a protocol. 622 is 9954/16: an HTTP allocation floor divided by a payload size, which is arithmetic on the workload rather than a property of anything.

And for a long time the harness did not compile. `zaphttp` moved from `net/http` to `fasthttp` objects, `NewTransport` became `Dial`, and `MarshalRequest` changed signature. Checked out at the commit contemporaneous with the published numbers, the harness still fails to build against today’s `zap-proto/http`:

```
cannot use http.HandlerFunc(echoHandler)
as fasthttp.RequestHandler
```

A benchmark that does not build is a benchmark nobody can check, which is how the first two problems survived. A later commit did migrate it to `fasthttp` and made the two HTTP arms share one handler — but it kept the byte echo as the native arm, so the second problem survived the migration that fixed the third.

We therefore finished the job rather than deleting the claim. The native arm is now a real ZAP message built with the published `zap-proto/go` runtime (§2); a `net/http` arm was restored, because that is the baseline the published claim was made against; a matched `fasthttp` arm sits between them so the transport comparison is not confounded by the object model; and everything was re-measured. The results that follow are worse for the native arm than what was published and better for the adapter. We keep the byte echo as an explicitly labelled lower bound, because the gap between it and the native arm is exactly the quantity the previous revision hid.

2 Five arms

net/http Go stdlib client and server, keep-alive, connection pooling. The everywhere baseline, and the only arm whose library offers no request pooling: it builds a fresh `http.Request` per call because there is no other way to use it.

fasthttp `fasthttp` client and server over HTTP/1.1, requests and responses acquired from and released to its pools. Same object model as the ZAP-HTTP arm, same wire as `net/http`. This arm exists so that the difference between it and ZAP-HTTP is the wire and nothing else.

ZAP-HTTP `zap-proto/http`: HTTP request and response semantics on the ZAP wire, driving the identical `fasthttp` handler.

Native ZAP A typed `zap-proto/go` v1.8.3 message carrying the body as a `bytes` field, the header set as a variable-entry list, and a status word, over 4-byte length-prefixed frames. The server parses the request, reads the body and every header, and builds a response echoing the body and the `X-` headers — the same work the HTTP handler does. Reads are zero-copy views into the frame; each end reuses one builder per connection.

Floor The same framing carrying raw bytes, echoed without being read. No message format, no schema, no headers, no dispatch. This is a lower bound on transport cost and is not a protocol; no number from this arm describes ZAP.

3 Methodology

Hardware and conditions. Apple M1 Max, 10 cores, macOS Darwin 25.6.0, Go 1.26.5, loopback only. **The machine was not quiet:** `run.sh` records `uptime` before and after each run, and the stored run began at a one-minute load average of 102.2 and ended at 102.1, on ten cores. Allocation counts and wire sizes are counts and do not move with load — they repeat across three repetitions, and the `net/http` allocation count reproduces the May 2026 run on the same machine to within 0.1% (115.4 vs. 115.3 allocations per request at the 16-byte workload). Throughput is a rate and does move. The widest swing in the stored run is the floor arm on the medium body: 20 590, 3 717, 5 187 `req/s` across 3 repetitions of the identical loop, a factor of 5.5. We report the rates, we report the load they were taken under, and we draw no conclusion from the sequential ones.

Workloads. Five bodies: tiny (16 B, 4 headers), small (256 B, 8), chat (824 B of chat-completion-shaped JSON, 12), medium (4 KiB, 12), large (64 KiB, 8). Echo handler, warm connection. The chat body is the shape the transport is actually deployed under; the other four span the range around it.

Driver. 50 warm-up requests, two forced `runtime.GC()`, then 5 000 requests with `MemStats` snapshotted around the window; 3 repetitions, medians reported. That snapshot is process-wide, so every figure covers the client and the server together — a round trip, not one end.

Fairness, and one asymmetry we did not remove. Every arm ships the same body and the same header set over a warm connection and consumes the whole response. Each client is driven the way its library is meant to be driven, which is itself part of what is measured. One asymmetry is deliberate: the native arm flattens its header set to bytes once, outside the request loop, while the HTTP arms walk a string-keyed map on every call. A typed protocol has no such map, and charging it for one would be charging it for a data structure it does not have.

Reproduce.

```
git clone https://github.com/zap-proto/bench
cd bench && ./run.sh > bench-results.txt 2>&1
```

The file in the repository is that command's complete output, machine line included. A claim with no matching line in it has no evidence behind it.

Workload	<code>net/http</code>	<code>fasthttp</code>	ZAP-HTTP	Native ZAP	Floor
tiny (16 B)	10 399	6.7	2.8	224.0	0.0
small (256 B)	10 449	6.1	3.2	704.1	0.0
chat (824 B)	14 681	8.5	4.4	1 984	0.0
medium (4 KiB)	24 748	11.6	6.9	8 384	0.0
large (64 KiB)	191 015	72.5	68.5	131 268	0.0

Table 1: Bytes allocated per round trip, `MemStats.TotalAlloc` delta over 5 000 requests, median of 3. `net/http` carries a ≈ 10 KiB floor independent of payload; the two pooled HTTP-object arms carry effectively none; the native arm carries twice the payload (§5).

4 Results

4.1 Bytes allocated per request

Two readings. Against `net/http`, ZAP-HTTP’s advantage is enormous and mostly uninformative: dividing 10 399 by 2.8 gives a four-digit ratio whose denominator is a rounding artifact, which is the same mistake $622\times$ was. Report the absolutes. Against the matched `fasthttp` baseline the honest statement is small and solid: ZAP-HTTP allocates fewer bytes per request than `fasthttp` on the identical handler at every size — 2.8 against 6.7 at 16 bytes, 68.5 against 72.5 at 64 KiB. Both are allocation-free in steady state; the difference is residual, and the wire is not costing anything.

4.2 Allocations per request

Workload	<code>net/http</code>	<code>fasthttp</code>	ZAP-HTTP	Native ZAP	Floor
tiny	115.4	0.0	0.0	6.0	0.0
small	122.4	0.0	0.0	6.0	0.0
chat	151.5	0.0	0.0	6.0	0.0
medium	158.9	0.0	0.0	6.0	0.0
large	143.1	0.0	0.0	6.0	0.0

Table 2: Heap allocations per round trip, median of 3; identical in all 3 repetitions to one decimal place.

`net/http` makes 115.4–158.9 allocations per request and the count barely depends on payload size: it is the cost of the request object, the URL, the header map, and the body reader. The native arm makes six, against the one this paper previously claimed.

The zeros want reading carefully. The harness reports one decimal place, so a `fasthttp` or ZAP-HTTP zero means *below 0.05 allocations per request* — one allocation every twenty or more requests, not none ever. The residual bytes in the previous table say the same thing: a few bytes per request amortized over 5 000 is a handful of allocations across the whole window, not a per-request cost. “Allocation-free in steady state” is the accurate claim; “zero allocations, ever” is not, and a table with a decimal place in it should not be read as making the second one.

Workload	net/http	fasthttp	ZAP-HTTP	Native ZAP	Floor
tiny	33	0	0	0	0
small	34	0	0	1	0
chat	50	0	0	4	0
medium	91	0	0	12	0
large	513	0	0	150	0

Table 3: GC cycles during the measured window, median of 3.

4.3 Garbage collection over 5 000 requests

The zero column is the operationally interesting one. Allocation rate drives GC frequency and GC frequency drives tail latency under load, and both pooled HTTP arms reach zero cycles at every payload size. The native arm does not: at 64 KiB it triggers a third as many cycles as **net/http**, not none.

4.4 Wire bytes

Workload	HTTP/1.1	ZAP-HTTP	Native ZAP
tiny (16 B)	243	272 (+11.9%)	212 (−12.8%)
small (256 B)	562	638 (+13.5%)	580 (+3.2%)
chat (824 B)	1 340	1 432 (+6.9%)	1 372 (+2.4%)
medium (4 KiB)	4 613	4 704 (+2.0%)	4 644 (+0.7%)
large (64 KiB)	65 844	65 918 (+0.1%)	65 860 (+0.0%)

Table 4: One serialized request, in bytes. Percentages are against the HTTP/1.1 column.

ZAP is not a wire-size win on this workload. The ZAP-HTTP frame is larger than the equivalent HTTP/1.1 request at every size — +0.1 to +13.5%, worst where the headers dominate and converging to parity as the body does. The native message is the only one that ever comes out smaller, and only at the 16-byte workload (−12.8%), where it drops the request line and the header names it does not need; above that it runs up to +3.2% larger, because each header entry carries a 4-byte length prefix where HTTP spends a colon and a newline. On the one realistic body here — the 824-byte chat payload — ZAP-HTTP costs 92 extra bytes on the wire and native ZAP 32.

This is a substantial improvement on the previous revision, which measured the ZAP-HTTP frame at 452 bytes against HTTP’s 243 — 86% larger — and printed it as “0.54x (−86.0% smaller)”. The codec has tightened from +86% to +11.9% at the same workload. The reporting bug is fixed too: the harness now names the direction, so a ratio cannot be read backwards.

4.5 Throughput

The stored run measured no throughput. The harness refuses to time anything on a machine whose load average exceeds its core count, and this run began at 102.2 on ten cores, so both rate sections read *NOT MEASURED* and there is no table here. That is the correct behaviour and the reason the section is empty rather than populated from an older run: a rate measured under that much contention describes the contention.

What a quiet run of this harness has shown, and what we would expect it to show again: both ZAP arms clear `net/http` by several times in every paired repetition under concurrency, while against the matched `fasthttp` baseline the paired ratios straddle 1.0 in both directions. The first is a gap an allocation profile predicts and that no repetition inverted; the second is not a result. Neither is quoted as a number here, because this run has none.

We make no claim at all from the sequential rates either. The previous revision’s 1.85–3.68× throughput claim, and its bandwidth table (1233 MB/s native against HTTP’s 519), are neither confirmed nor refuted here; they are unmeasurable on this machine in this state. We withdraw them rather than restate them from output that cannot carry them, and they want a re-run on an idle host.

5 Where the native arm’s allocations are

The native figures are not noise and they have an exact form. Writing B for the body size, the measured bytes per round trip are

$$224.0 = 2B + 192, \quad 704.1 = 2B + 192, \quad 8384 = 2B + 192, \quad 131268 = 2B + 196$$

at $B = 16, 256, 4096, 65536$ — two copies of the payload plus a fixed 192 bytes of builder scaffolding, at every size, in six allocations. (At 64 KiB the constant reads 192 to 196 across repetitions; the other three repeat to the byte.)

The chat workload confirms the mechanism rather than breaking it. That body is 824 bytes and the measured figure is 1984, not $2(824) + 192$. The excess over the constant is 336 rather than 192, and the difference between them is twice the gap from 824 to 896 — Go’s next slice size class. The copy is a slice allocation, so it rounds up: the relation is two *allocations* of the payload, not two copies of exactly B bytes. Two of them, for a round trip with two encodes, is one heap copy per encode.

The source says the same thing. `ObjectBuilder.SetBytes` in `zap-proto/go` v1.8.3 defers a variable-length field by storing `append([]byte(nil), v...)` — a fresh heap allocation holding the whole payload — and `Finish` then copies that buffer a second time into the message. The builder’s own buffer is reused across requests and does not grow after warm-up; the per-request cost is the deferred copy.

The floor arm settles what this is not. It runs the same framing over the same buffered connection and reports 0.0 bytes per request at every size, on the same one-decimal scale that makes the pooled HTTP arms read as zero. So the $2B$ is the message format’s encode path, not the framing and not the socket.

The fix is mechanical rather than structural — append directly into the builder’s buffer and patch the offset afterwards, which is what the deferred entry already does one step later — and it is in neither v1.1.0 nor the current v1.8.3. We first measured this against v1.1.0 and re-measured against v1.8.3 when the harness was bumped: $2B + 192$ and six allocations, to the byte, on both. It is a property of the released runtime rather than of a stale tag, and an independent measurement of the same code path in a third tag reports it too [3].

6 What this paper does not measure

Authentication and post-quantum transport. The benchmark runs without TLS or any handshake. ZAP’s hybrid KEM [2] is a connection-setup cost amortized across a warm connection, but we have not measured it, and nothing here is a statement about it.

A quiet machine. Everything in §1 about the previous revision applies to us if we report a rate taken at load 102 as though it were a hardware capability. The allocation and wire columns stand on their own; the throughput columns want a re-run on an idle host and should be read as provisional until they get one.

A real network. Loopback removes packet loss, the NIC, and the kernel’s receive path. Allocation counts are indifferent to all three; throughput is not, and on a saturated link the codec stops being the bottleneck at all [3].

Multiplexed streams. Sequential single-connection RPC only. The case where a binary transport has structural room that HTTP/1.1 does not — thousands of logical streams over one connection — is `zap-proto/ws`, and is not exercised here.

7 Conclusion

On the identical handler and against a baseline with the same object model, ZAP-HTTP is allocation-free per request, allocates fewer bytes than `fasthttp` at every payload size, triggers no garbage collection over 5 000 requests, and carries a frame +0.1–+13.5% larger than the equivalent HTTP/1.1 request. Against Go’s `net/http` it removes a 10 KiB, 115.4-allocation per-request floor. That floor is the number a `stdlib` service would actually see, and it is worth more than the four-digit ratio the floor produces when divided by a rounding artifact. Against `fasthttp`, which already has no such floor, the throughput margin is not resolvable on this machine and we do not claim one.

The native path is where this paper was wrong and is now worse than advertised. Built on the published runtime it allocates six objects and two heap copies of the payload per round trip, not one object and the payload exactly, and against `fasthttp` its throughput straddles parity in both directions. The 622×, 115× and 1.85–3.68× figures this paper carried are withdrawn: the first two were arithmetic on a payload size against an arm that was not ZAP, and the third was never measurable from the output on disk. What replaces them is smaller, and it is in the file.

References

- [1] ZAP Protocol Authors. *Zero-Copy Throughput of the ZAP Wire Format*. `zap-proto/papers`, 2026. [zap-proto/papers/zero-copy-throughput](#).
- [2] ZAP Protocol Authors. *Z-Wing: A Triple-Hybrid KEM for the ZAP Handshake*. `zap-proto/papers`, 2026. [zap-proto/papers/pq-hybrid-kem](#).
- [3] Hanzo AI Research. *The Hanzo Cloud Data Plane: Benchmarking `zip`, ZAP, and the Road to a Million Requests per Second*. 2026. Independently reports the same deferred-tail copy in a later `zap-proto/go` tag, and reports both transports tying once a 2.5 GbE link saturates.
- [4] R. Fielding, M. Nottingham, and J. Reschke. *HTTP Semantics*. RFC 9110, June 2022.